

# An fMRI study of code review and expertise

Nikhil Anand

February 11, 2026

This paper studies how the brain processes computer code compared to natural language using fMRI. The researchers asked 29 participants to perform three tasks: code comprehension, code review, and prose (English text) review, while their brain activity was recorded. *They found that programming languages and natural language are represented differently in the brain.* Using machine learning on brain activity data, they could classify which task a participant was performing with high accuracy. Interestingly, the same brain regions were involved in distinguishing code from prose across tasks. *The study also found that expertise matters: more experienced programmers showed less difference in brain activity between code and prose tasks.* This suggests that as programmers gain skill, their brains may process code more similarly to natural language.

## 1 Motivation

The central motivation of this paper is that software engineering heavily depends on human judgment, yet we know very little about how the brain actually processes programming tasks. Activities like code comprehension and code review are fundamental to software development, but research in this area has mostly relied on behavioral studies, surveys, performance metrics, or self-reports. These methods tell us what developers do, but not how their brains represent and process code. The authors argue that this is a major gap. Developers spend a large portion of their time understanding code, and companies like Google and Facebook require formal code review for new contributions. Code understanding is often ranked as more important than even functional correctness in real-world software contexts. Despite this practical importance, there is almost no neuroscientific understanding of how programming is represented in the brain.

Another key motivation is the relationship between programming languages and natural languages. Prior research has shown that software has statistical properties similar to natural language. This raises an important question: does the brain process code like it processes English? Or does programming rely on completely different neural systems? Understanding this distinction could influence how we teach programming, how we design tools, and how we conceptualize programming as a cognitive activity.

The paper is also motivated by expertise. There are well-documented productivity differences between novice and expert programmers, sometimes by an order of magnitude. Other fields like physics, music, or taxi driving have shown that expertise can physically alter brain organization. However, no prior work had examined whether programming expertise changes neural representations. The authors aim to explore whether more experienced programmers process code differently at a neural level.

Beyond curiosity, the authors outline several broader reasons for using medical imaging in software engineering:

- Self-reports are often unreliable; brain data could provide more objective insight.
- Understanding neural processing could improve pedagogy by clarifying whether programming is cognitively closer to language, math, or something else.
- It may help explain how expertise develops and why experts are more efficient.
- It could guide tool design by revealing how humans mentally evaluate code.

In short, the paper is motivated by a fundamental question: what is the neural basis of programming? By using fMRI, the authors aim to move beyond behavioral performance and directly examine how code and prose are represented in the brain, and how that representation changes with expertise.

## 1.1 Background

*Most of the background is quoted from the paper.*

### 1.1.1 Code Review

Static program analysis methods aim to find defects in software and often focus on discovering those defects very early in the codes lifecycle. Code review is one of the most commonly deployed forms of static analysis in use today; well-known companies such as Microsoft, Facebook, and Google employ code review on a regular basis. At its core, code review is the process of developers reviewing and evaluating source code. Typically, the reviewers are someone other than author of the code under inspection. Code review is often employed before newly-written code can be committed to a larger code base. Reviewers may be tasked to check for style and maintainability deficiencies as well as defects.

### 1.1.2 Code comprehension

Much research, both recent and established, has argued that reading and comprehending code play a large role in software maintenance. A well-known example is Knuth, who viewed this as essential to his notion of Literate Programming. He argued that a program should be viewed as *“a piece of literature, addressed to human beings”* and that a readable program is *“more robust, more portable, [and] more easily maintained.”* Knight and Myers argued that a source-level check for readability improves portability, maintainability and reusability and should thus be a first-class phase of software inspection.

### 1.1.3 Computer Science and Neuroscience

Siegmund et al. presented the first publication to use fMRI to study a computer science task: code comprehension. The focus of their work was more on the novelty of the approach and the evaluation of code comprehension and not on controlled experiments comparing code and prose reasoning.

Some researchers have begun to investigate the use of functional near-infrared spectroscopy (fNIRS) to measure programmer blood flow during code comprehension tasks. In smaller studies involving eleven or fewer participants, Nakagawa et al. found that programmers show higher cerebral blood flow when analyzing obfuscated code and Ikutani and Uwano found higher blood flow when analyzing code that required memorizing variables. fNIRS is a low-cost, non-invasive method to estimate regional brain activity, but it has relatively weak spatial resolution (only signals near the cortical surface can be measured); the information that can be gained from it is thus limited compared to fMRI. These studies provide additional evidence that medical imaging can be used to understand software engineering tasks, but neither of them consider code review or address expertise.

## 2 Methodology

### 2.1 fMRI

Functional magnetic resonance imaging (fMRI) is a non-invasive neuroimaging technique that measures brain activity by detecting changes in blood flow. When a specific brain region is more active, it consumes more oxygen, leading to increased blood flow to that area. fMRI detects these changes in blood oxygenation levels, allowing researchers to infer which regions of the brain are involved in specific cognitive tasks. In this study, fMRI was used to compare brain activity during code comprehension, code review, and prose review tasks.<sup>1</sup>

### 2.2 Experiment Design

#### 2.2.1 Participants

Thirty-five students at the University of Virginia were recruited for this study. Solicitations were made via fliers in two engineering buildings and brief presentations in a third-year “Advanced Software Devel-

---

<sup>1</sup>The authors do point out that the use of fMRI in Software Engineering is still exploratory and that there are limitations to the technique

opment Techniques” class and a fourth-year “Language Design & Implementation“ class. Out of these 35 participants, 29 were included in the final analysis after excluding those with excessive head motion during scanning. All of the participants were,

- Right-handed
- Native English speakers
- Screened for programming experience and MRI safety

The final sample comprised of 18 males and 11 females. Additional objective and subjective measures of programming experience were collected, including years of programming experience, number of programming languages known and GPA.

### 2.2.2 Tasks

Participants performed three tasks while undergoing fMRI scanning:

- **Code Comprehension:** Participants read a short code snippet and answered a true/false question about its behavior.
- **Code Review:** The participants were shown a Github style pull request, code differences and a comment from a reviewer. They were asked to evaluate the comment and decide to accept or reject the pull request.
- **Prose Review:** Participants were shown a short English passage and suggested edits. They were asked to evaluate the edits and decide to accept or reject them.

## 2.3 Preprocessing

Before analysis, the raw brain data underwent standard preprocessing for motion correction (head movement adjustment), alignment to a standard brain template (MNI space), noise correction, and no spatial smoothing was applied. This ensured that the data was clean and comparable across participants, while preserving the fine-grained spatial information necessary for multivariate pattern analysis (MVPA).

## 2.4 Processing

The extremely large dimension of fMRI data is a major obstacle for machine learning — we commonly have tens of thousands of voxels<sup>2</sup> but only a few dozen training examples. This can be solved using a simple linear map (a kernel function) that reduces the dimensionality of the feature space. The training inputs were given as a set of vectors  $\{x_n\}_{n=1}^N$ , with corresponding labels  $\{y_n\}_{n=1}^N$  where  $N$  is the number of beta images for each participant across both classes. To learn a mapping, the authors employed a cumulative Gaussian, and specified posterior conditional over  $f$  using the Bayes rule

$$p(f|\mathcal{D}, \theta) = \frac{\mathcal{N}(f | 0, K)}{p(\mathcal{D} | \theta)} \prod_{n=1}^N \phi(y_n f_n)$$

where  $\mathcal{N}(f | 0, K)$  is a zero-mean prior,  $\phi$  is a factorization of the likelihood of training samples.

# 3 Key Findings

## 3.1 Research Question 1

RQ1: Can we classify which task a participant is performing based solely on patterns of their brain activity?

---

<sup>2</sup>A voxel is the smallest unit of an fMRI image, analogous to a pixel in a regular image.

Code Comprehension vs Prose tasks were highly distinguishable, with a 79% balanced accuracy ( $p < 0.001$ ) and Code Review vs Prose tasks were also highly distinguishable, with a 71% balanced accuracy ( $p < 0.001$ ). This strongly suggests that the brain forms distinct neural representations for programming tasks versus natural language tasks. Code is not processed identically to English prose and even though Code Review and Code Comprehension share similarities, but still differ at the neural level.

### 3.2 Research Question 2

RQ2: Can we relate these task differences to specific brain regions?

The same set of brain regions distinguished code review and comprehension from prose. The regional importance maps were almost perfectly correlated ( $r = 0.99, p < 0.001$ ). Highly involved regions included the prefrontal cortex (executive control, reasoning, decision-making) and the language-related areas near Wernicke’s area.

This means that code and prose differ consistently in how they activate higher-order cognitive regions. Programming is not processed purely as math or purely as language. It heavily recruits executive control regions (working memory, abstract reasoning). But language-related areas are also involved. This finding suggests programming is a hybrid cognitive activity, involving language systems and executive reasoning systems. And importantly, the same neural system distinguishes code from prose across different kinds of code tasks.

### 3.3 Research Question 3

RQ3: Does expertise influence neural representation of code vs prose?

This is one of the most interesting parts of the paper. The authors measured expertise using CS GPA (corrected for number of credits), then correlated it with classifier accuracy. There was a negative correlation between expertise and classification accuracy ( $r = -0.44, p = 0.016$ ) for code comprehension vs prose.

This means that as programmers become more skilled, their brain activity for code and prose becomes less distinguishable. Experts process code in a way that is more similar to natural language. With experience, code may become more “language-like” in neural representation. Expertise leads to more integrated cognitive processing. This aligns with findings from other domains (e.g., physics, music, taxi driving) where expertise reorganizes neural representations.

## 4 Implications

**Programming Has a Distinct Neural Basis** The finding that code and prose can be reliably classified from brain activity implies that programming is not simply “reading English with symbols.” It recruits distinct neural representations. This challenges the common assumption that programming is purely a linguistic activity. Instead, it appears to involve a unique cognitive configuration that blends language and executive reasoning systems. This has foundational implications: programming should be understood as its own cognitive domain, rather than being reduced to either language or mathematics. It justifies studying programming cognition as a standalone scientific field.

**Programming Is a Hybrid Cognitive Activity** Although code and prose are distinct, they activate overlapping language-related regions along with executive control regions (e.g., prefrontal cortex). This suggests programming is cognitively hybrid — it involves structured language processing but also heavy working memory, abstraction, planning, and decision-making. The implication here is that programming education and research should not frame coding as purely syntactic or linguistic. Instead, it requires integrated training in reasoning, control processes, and structured abstraction. This supports the idea that difficulty in programming may arise not from syntax learning, but from cognitive load and executive demand.

**Expertise Changes Neural Representation** One of the most profound implications is that increased programming expertise reduces the neural distinction between code and prose. This suggests that with experience, code becomes cognitively more “natural.” Experts may internalize programming structures similarly to how fluent readers process language. This supports theories of neural plasticity — expertise reorganizes how information is represented in the brain. It implies that the dramatic productivity gap between novice and expert programmers may not just be skill-based, but neurologically grounded. Programming expertise appears to alter mental representation itself.

**Rethinking Code as “Natural Language”** Previous research showed software shares statistical properties with natural language. This paper adds neural evidence to the discussion. While code is statistically language-like, it is neurally distinct — especially for novices. However, expertise moves it closer to language representation. This nuanced finding suggests that programming may be “learned language,” rather than inherently language-like. It strengthens interdisciplinary connections between cognitive science, linguistics, and software engineering.